

My Story Marker

Novelas personalizadas para regalar, escritas por un harness de agentes que se puede verificar

10

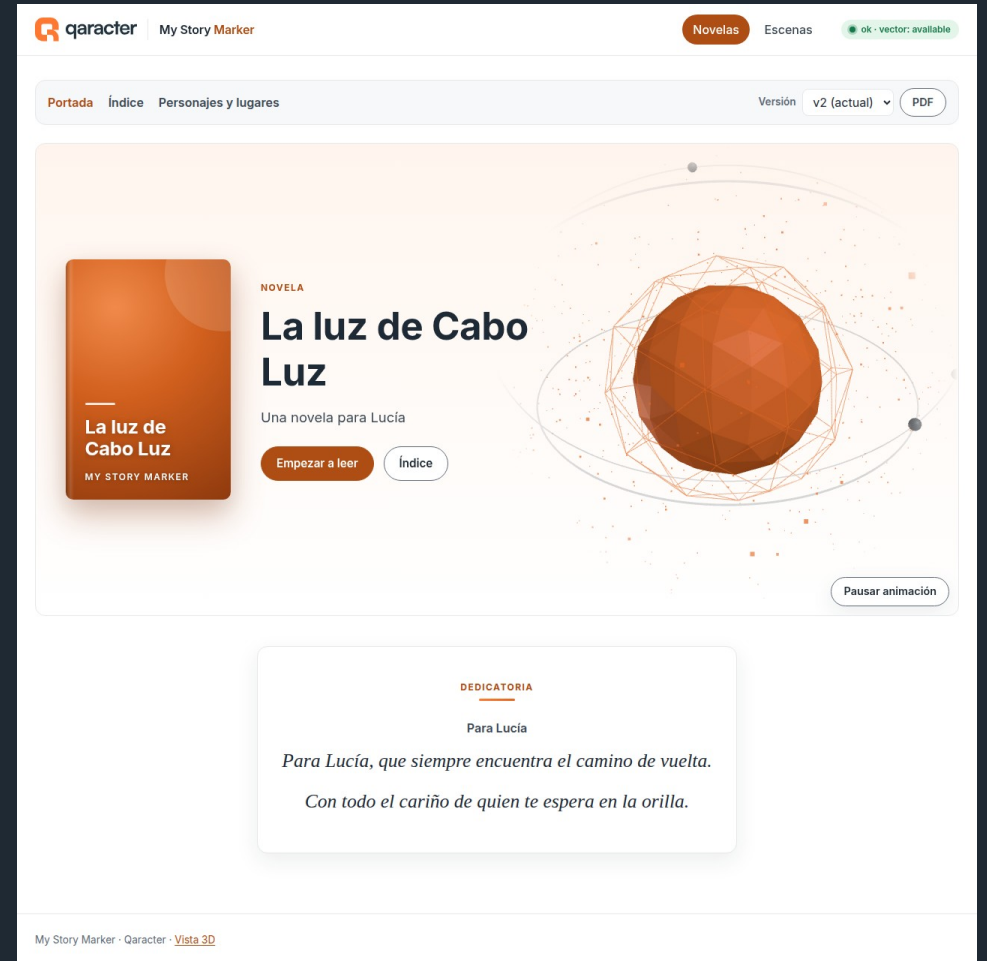
capítulos de 1.000–1.500 palabras, en español

6

roles con Claude Haiku 4.5 orquestados por código

2

verificaciones formales: Lean 4 y TLA+



Un regalo único que no se puede comprar hecho

El problema

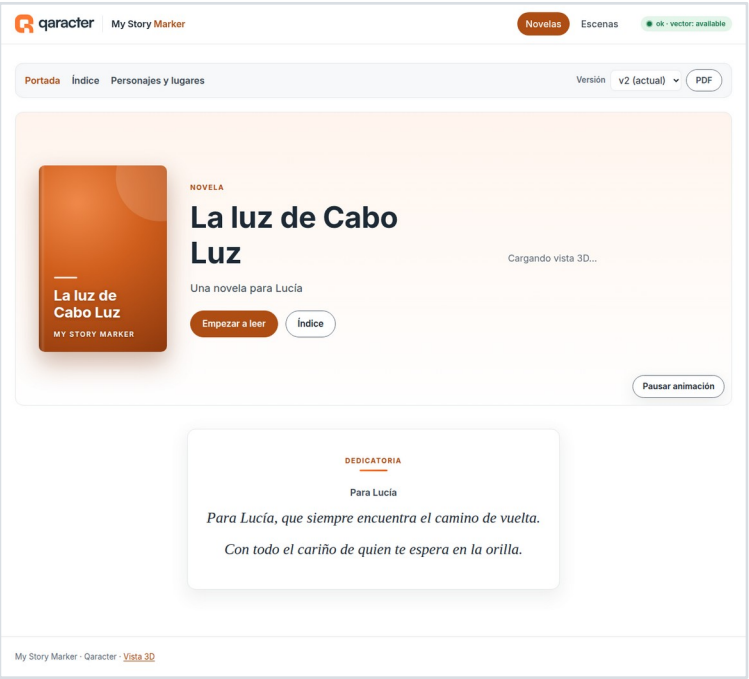
- **Escribir una novela sobre alguien** lleva meses; un regalo tiene fecha.
- **Un LLM sin harness** mezcla nombres, olvida recuerdos, contradice fechas y resucita a la mascota.
- **El cliente tiene vetos** (un nombre, un tema) que no pueden aparecer nunca.

La propuesta

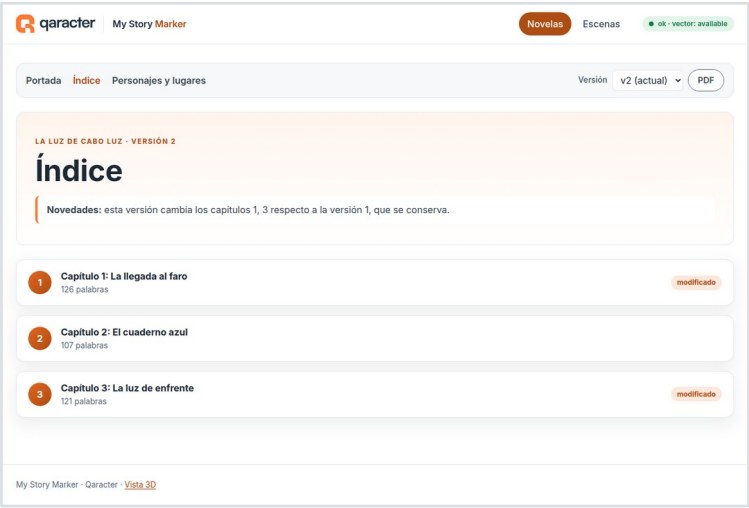
- **Entrevista guiada** → brief validado con schema; detecta datos que faltan y contradicciones.
- **Novela de 10 capítulos** en la que el destinatario se reconoce.
- **Lectura web y PDF:** portada con dedicatoria, índice, fichas de personajes.
- **Cambios puntuales:** «el perro se llama Nala» regenera solo lo necesario.

D11 • Personalización y calidad narrativa pesan igual. Que aparezcan todos los datos no basta si la historia no funciona como historia: validadores, editor y juez comprueban las dos cosas.

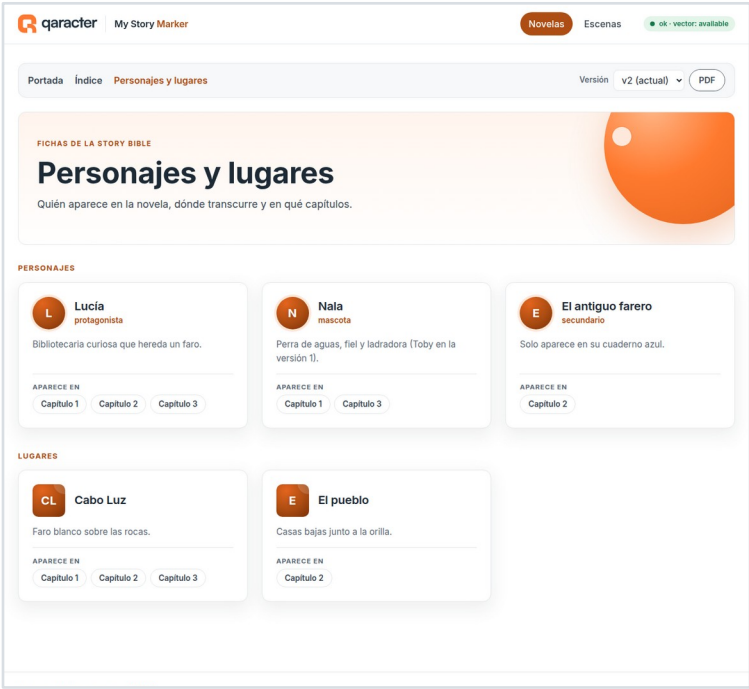
Del brief a una novela que se lee y se regala



1 Portada con dedicatoria personalizada



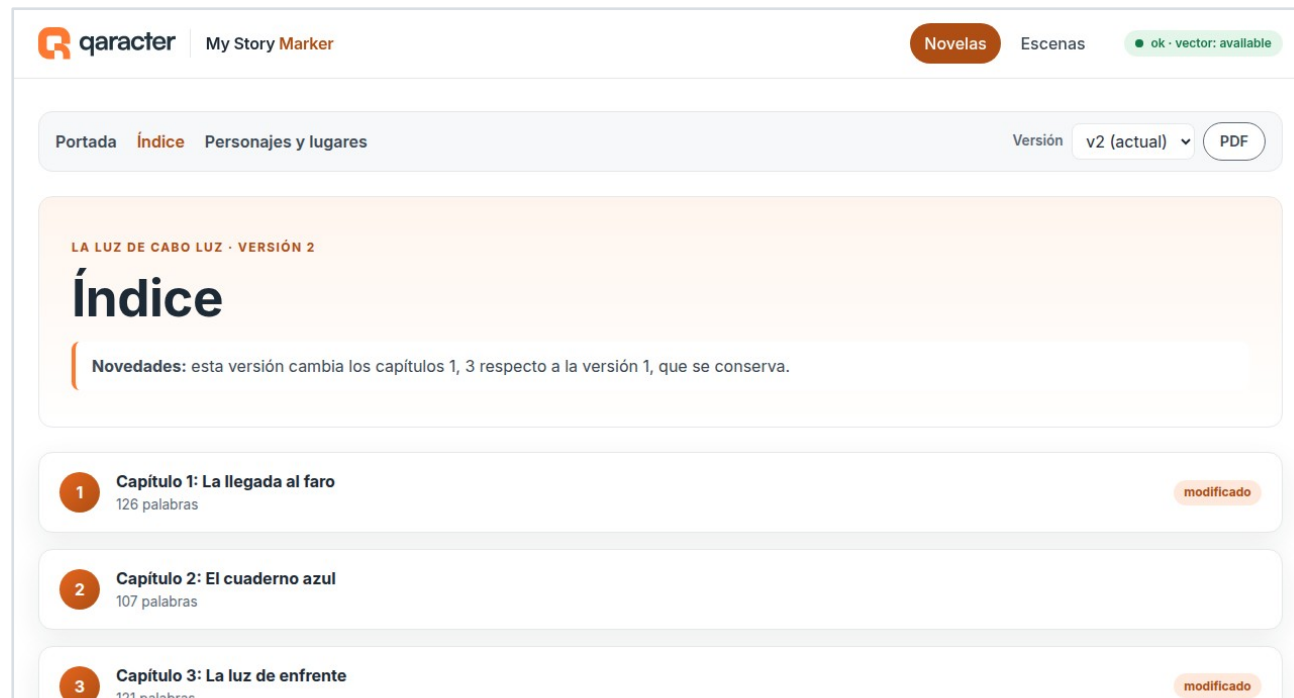
2 Índice navegable con marcas de versión



3 Fichas desde la story bible, con «Aparece en»

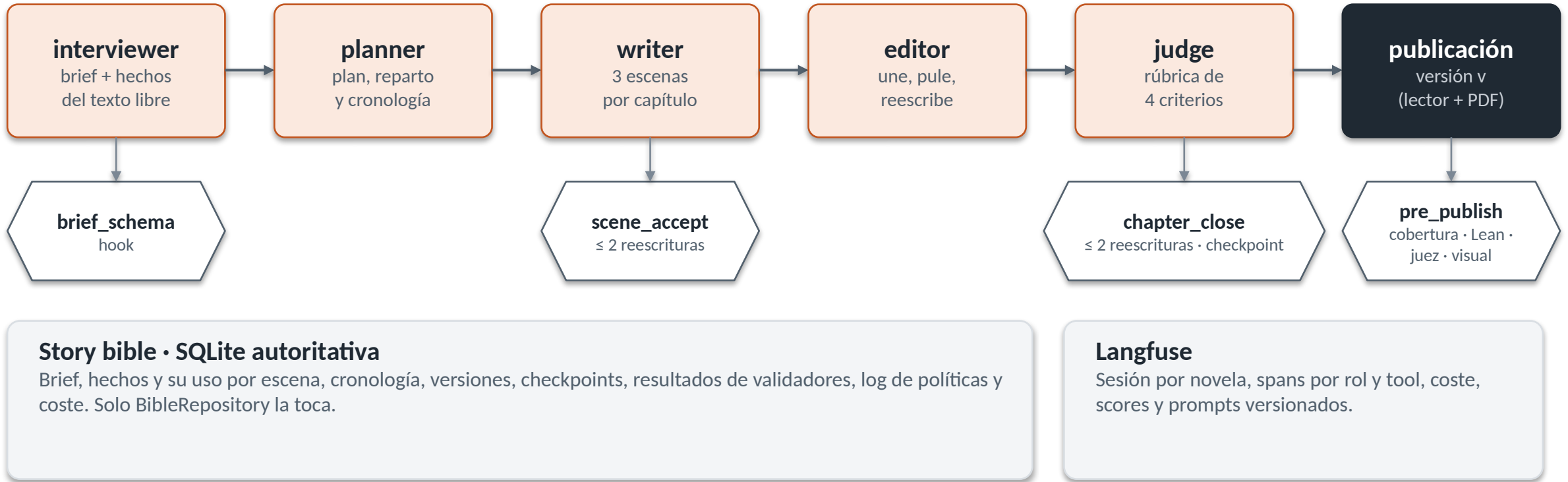
El lector pide un cambio: «el perro se llama Nala»

- 1 Selecciona el hecho**
en la página o por CLI/API (change_fact).
- 2 Se actualiza la story bible**
el hecho es único: nunca hay dos versiones de él.
- 3 Solo se regeneran sus capítulos**
fact_usage dice qué capítulos lo usan.
- 4 Versión v+1 publicada**
la v1 se conserva; el índice marca «modificado».
- 5 PDF nuevo con «novedades»**
capítulos cambiados con enlaces internos.



Ejecución real (ab4731f): novela de 2 capítulos → versión 2 publicada, 0 apariciones del nombre antiguo, 16 del nuevo, versión 1 intacta.

Roles con un solo trabajo, orquestados por código



Ningún modelo tiene herramientas. El orquestador (`app/novel/pipeline.py`) decide qué documentos ve cada rol, valida su salida con JSON Schema y escribe en la BD en su nombre: la tabla de permisos se aplica en código, no en un prompt. Cada fallo vuelve, con feedback y presupuesto, al rol que puede arreglarlo.

La story bible: una SQLite autoritativa con 16 tablas

1

Hechos y su uso

`fact` · `fact_usage`

Cada hecho del brief es una fila; `fact_usage` lo liga a (versión, capítulo, escena). Así `change_fact` sabe qué capítulos regenerar.

2

Cronología

`chronology_event` · `event_participant`

Fechas, lugares, participantes, edades declaradas y salidas (muerte, partida). Es la entrada del modelo Lean.

3

Versiones y checkpoints

`novel_version` · `chapter_version` · `checkpoint`

Texto + hash por capítulo y versión; texto y checkpoint en una transacción. Reanudación en el primer capítulo incompleto.

4

Auditoría y coste

`validator_result` · `policy_decision` · `llm_call`

Cada validador, decisión de política y llamada con tokens, coste, latencia y versión de prompt. Nada se borra (D6).

Diagrama entidad-relación completo (Mermaid) en anexo-arquitectura.pdf

Cada fallo se detecta donde es barato repararlo



Tipo	Validadores	Punto de ejecución	Si falla
Programático	brief_schema · schema_role_output · chapter_length · exact_names · brief_coverage · prose_repetition · no_placeholders · visual_check	hook · scene_accept · chapter_close · pre_publish	vuelve al rol productor con la evidencia
Guardrail	forbidden_words_scene · forbidden_words_chapter · free_text_injection (prescan)	entrevista · scene_accept · chapter_close	reescritura ≤ 2 → forbidden_word_limit
Semántico	judge_chapter · judge_novel · revisión humana (misma rúbrica)	chapter_close · pre_publish · una novela	editor; aprueba si cada criterio ≥ 3 y media ≥ 3,5
Formal	lean_chronology (4 invariantes) · TLA+/TLC del flujo (en desarrollo)	pre_publish · desarrollo	versión bloqueada; 1 ronda de reparación de los capítulos citados

Cada validador tiene nombre y punto de ejecución, y envía su resultado a Langfuse como score. **Un fallo en pre_publish no reescribe la novela:** solo se reabren los capítulos que cita la evidencia.

Reglas que se cumplen aunque el modelo no quiera

Vetos en 3 niveles

- **Global:** sembrado por migración (1300).
- **Novela:** los vetos del cliente en el brief.
- **Léxico:** variantes extra que aporta cada validador.
- **Un acierto** vuelve al escritor, ≤ 2 veces.

Un solo normalizador

- **Plegado común** de texto y término (NFKD, sin tildes).
- **Detecta:** t0nt0 · tontooo · t.o.n.t.o · cabrones → cabrón.
- **Sin falsos positivos:** «ridículo», «tontería».
- **Palabra completa** y frases, no subcadenas.

Audit log y hooks

- **policy_decision:** cada decisión con término, intento y traza.
- **Score guardrail:*** en Langfuse.
- **PreToolUse policy_guard:** .env, claves, escritura directa a la BD.
- **PostToolUse validate_chapter:** mismas reglas en ediciones a mano.

Intento en Claude Code (commit 54854d3)	Resultado
Write backend/.env · valor con forma de clave sk-ant-...	bloqueado (exit 2)
Bash sqlite3 data/harness.sqlite "delete ..."	bloqueado (exit 2)
Bash sqlite3 ... "select ..." · Edit backend/app/main.py	permitido (exit 0)
Capítulo con «c4br0n» o de 2 palabras	bloqueado (exit 2)

Un build verde prueba que la cronología es coherente

temporalOrder

Lo que se cuenta después ocurre el mismo día o más tarde.

agesCoherent

Cada edad declarada = años cumplidos desde el nacimiento.

noBiLocation

Nadie está en dos lugares el mismo día.

noAfterExit

Tras una muerte o partida, esa persona no vuelve a aparecer.

3 s

build • 100 eventos

17 s

build • 300 eventos

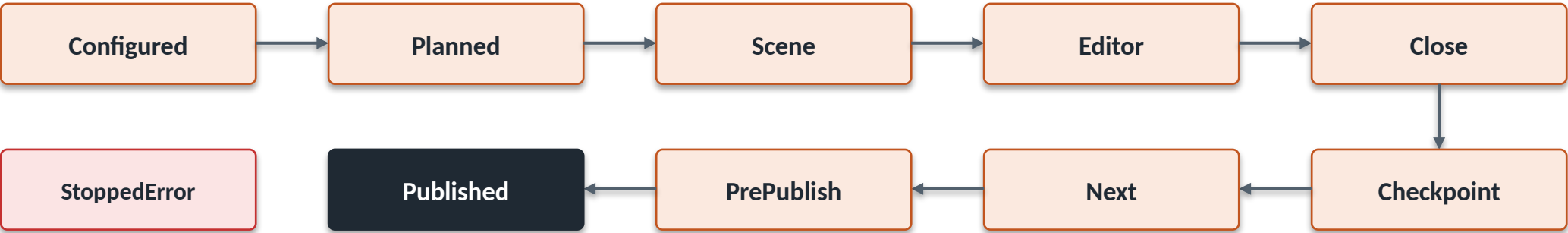
- **Bool + Prop + teorema de corrección:** la comprobación es una prueba, no un true.
- **Solo core Lean, sin Mathlib;** decide +kernel (decide revienta maxRecDepth ~100 eventos).
- **Se genera desde la story bible** en pre_publish; si falla, no se publica y el fallo vuelve al editor con los eventos.

Caso real

b4-temporal: el diagnóstico de cronología del plan (los mismos invariantes) detectó noAfterExit — la mascota muerta que reaparece — antes de escribir una línea; tras 1 replan, parada controlada (0,25 USD en vez de una novela entera).

Pendiente: un caso cazado por Lean en pre_publish y por ningún otro validador (docs/process/lean-caso-real.md).

El flujo del harness, comprobado con TLC



Bucles acotados (≤ 2 reintentos, 1 reparación) $\rightarrow$ StoppedError al agotarse. Crash $\rightarrow$ Resume desde la BD. ChangeFact $\rightarrow$ versión v+1.

Propiedad	Tipo	Garantiza
NoUnvalidatedPublish	invariante	no se publica nada que no pasó todos los validadores
ResumeNoDupNoLoss	invariante	reanudar no duplica ni pierde capítulos
PreviousVersionKept	invariante	la versión publicada anterior no cambia
RetriesBounded	invariante	reintentos acotados, también tras un crash
Termination • EveryGenerationEnds	liveness	toda generación acaba publicada o en error

Sin errores

696.062

estados distintos

1.247.479 generados • profundidad 109 • 59s

N = 5 capítulos • 3 escenas • reintentos 2/2

Cuatro trazas de TLC que cambiaron el código

El modelo se escribió en paralelo al pipeline, leyendo el esquema y el plan tal cual. TLC encontró dónde esa lectura se rompía.

	Invariante violado	Traza	Causa	Regla para el código
CE1	ResumeNoDupNoLoss	18 estados	Un crash entre la fila del capítulo y su checkpoint duplica el capítulo	Texto del capítulo y checkpoint en una sola transacción
CE2	RetriesBounded	21 estados	El contador de reintentos de capítulo vivía en memoria; un crash lo reiniciaba	Reintentos de capítulo contados desde validator_result persistidos
CE3	ResumeNoDupNoLoss	46 estados	La ronda de reparación insertaba una segunda fila del mismo capítulo	Upsert por (versión, capítulo); una versión publicada no se escribe
CE4	RetriesBounded	50 estados	El contador de rondas de reparación vivía en memoria; un crash lo reiniciaba	Ronda de reparación guardada en novel_version junto a «blocked»

Efecto: la spec de la story bible volvió a borrador y se reprobó antes de que existiera el pipeline, que nació con las cuatro reglas. Dos mutaciones (publicar sin pre_publish, regenerar en sitio) confirman que las propiedades no son vacías.

Qué hizo cada llamada, cuánto costó y con qué prompt



Prompts versionados en las evals

Rol	Prompt	Versión	Llamadas
editor	editor	3	12
interviewer	fact_extractor	sha-609af91...	1
judge	judge_chapter	1	11
judge	judge_novel	1	4
planner	planner	2	4
writer	writer	3	18

Cada llamada guarda prompt_version también en llm_call (SQLite), así el coste y la comparación antes/después no dependen de Langfuse. Sin claves: NoopObserver.

Cinco briefs × validadores · iteración «before»

brief	schema	vetos esc.	vetos cap.	longitud	nombres	cobertura	repetición	juez cap.	juez nov.	Lean	visual	inyección	final	USD
b2-infantil	✓	✓	✓	✓	✓	✗	✓	✓	✓	✓	✓	—	blocked v1	1.1247
b3-injection	✓	✓	✓	✓	✓	✗	✓	✓	✓	✓	✓	🚩	blocked v1	1.2392
b4-temporal	✓	—	—	—	—	—	—	—	—	—	—	—	pipeline_error	0.2494
b5-contradiction	✗	—	—	—	—	—	—	—	—	—	—	—	rejected_by_validation	—

Iteración de tuning

Lo que ya muestra «before»: el validador que bloquea es brief_coverage (recuerdos obligatorios que no aparecen literalmente); b3 marca la inyección y no la sigue; b4 se detiene en el plan por la cronología; b5 se rechaza antes de generar, como se esperaba.

Pendiente: la ejecución «after» tras un cambio de prompt y su comparación (compare_iterations.py → evals/results/tuning.md); esta sección se rellena sola al reconstruir.

Ataques probados, qué los detuvo y qué queda pendiente

#	Caso	Defensa	Estado
R1	Inyección en texto libre (vivo, B2)	prescan + indicador del modelo; solo hechos al bible	✓ detectado y no seguido
R2	Inyección en el brief b3-injection	prescan, extractor, forbidden_words_*, juez	prescan ✓ (validación estática); generación pendiente
R3	Trampas temporales b4-temporal	lean_chronology (4 invariantes), chronology_problems en el plan, judge_novel	pendiente
R4	Contradicción b5-contradiction	validación del brief (brief_schema)	✓ rechazado, no se genera
R5	Variantes de palabras prohibidas	normalizador de app/policy/normalise.py	✓ tests; en prosa generada pendiente
R6	Fuga de secretos vía Claude Code	hook policy_guard.py	✓ bloqueado (exit 2)
R7	Escritura directa en la story bible	hook policy_guard.py	✓ bloqueado; lectura select permitida
R8	Nombres anonimizados por el propio modelo	no_placeholders + prompts	✓ (ver iteraciones)

Hallazgo de revisión (R2): el texto libre crudo llegaba al planner y al juez; desde a721bc7 solo reciben los hechos extraídos.

Construido por un orquestador y subagentes en paralelo

1 CLAUDE.md + AGENTS.md

Mapa de docs y Procesos 0-3: docs → spec → plan → código. Proceso 0 en 4 rondas antes de la spec 004.

2 Subagentes por bloque

12 bloques, 4 oleadas, un git worktree por agente; contratos K1-K5 publicados antes que su implementación.

3 Skills

gift-novel-run (creada: generar e inspeccionar de punta a punta), react, sqlite, verification, fastapi.

4 Comandos /

/generate-novel • /inspect-novel • /change-fact
• /exam-gap.

5 Hooks

PreToolUse policy_guard y PostToolUse
validate_chapter: el mismo código que el
pipeline, 14 casos probados.

6 Browser MCP

Playwright MCP inspeccionó el lector: cazó un
500 intermitente (SQLite entre hilos) y fichas sin
versionar; ambos corregidos.

Un límite que funcionó: un subagente intentó ampliar las exenciones de una regla de seguridad y el clasificador del modo auto lo bloqueó; la decisión quedó para el humano.

Lo que cuesta una novela con Claude Haiku 4.5

0,31 USD

coste medio por capítulo

7 min

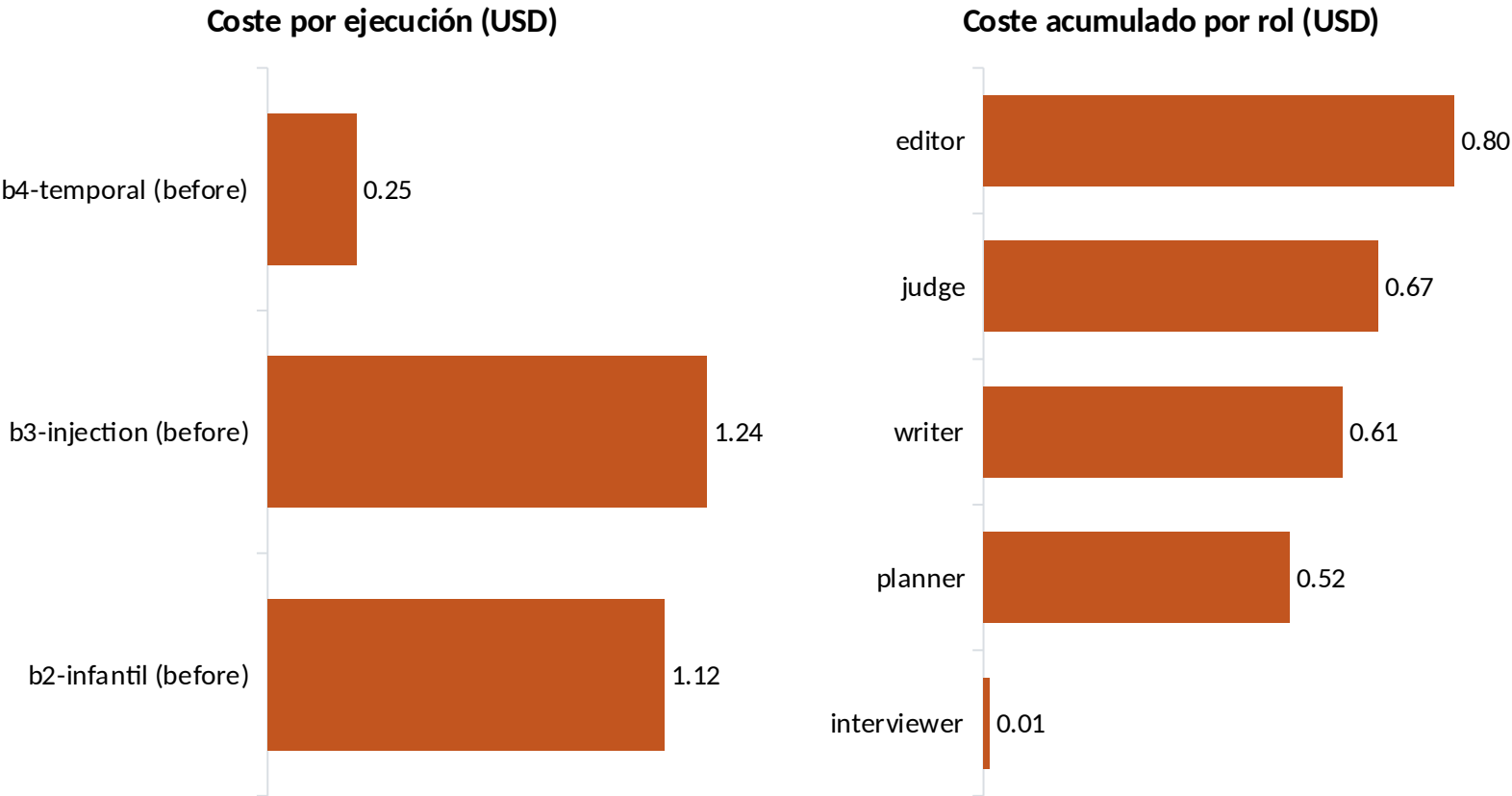
tiempo de modelo por capítulo

3,36 USD

estimación, novela de 10 capítulos

80 min

estimación de tiempo, 10 capítulos



Medido en 6 capítulos de 3 ejecuciones de evals (tiempo de modelo secuencial); smoke de 1 capítulo: 0,30 USD, 7 llamadas, ~6,5 min.

Qué elegimos, frente a qué, y lo que costó

Decisión	Frente a	Por qué	Coste
Multi-agente con orquestador determinista	un agente con herramientas	permisos en código, reintento por paso, trazas por rol	más llamadas y más orquestación
SQLite autoritativa (y el texto en la BD)	ficheros + índice derivado	una sola verdad para validadores, lector y Lean	sin diffs de git sobre la prosa
Lector web + PDF exportado	solo PDF interactivo	pedir cambios, ver versiones, validar con MCP	dos superficies que mantener
TLA+ antes que el pipeline	modelar después	4 errores de diseño cazados cuando eran baratos	el modelo abstrae prosa y validadores
Lean core, decide +kernel	Mathlib, native_decide	sin descargas; confía en el kernel	noBilocation cuadrático
claude -p sin tools	SDK con API key	ninguna clave en el repo; rol aislado	arranque por llamada; política de privacidad del CLI
Haiku 4.5 para todos los roles	Sonnet/Opus en writer y juez	coste y latencia de 10 capítulos	calidad vigilada por juez; subida por rol
Juez bloqueante (≥ 3 , media $\geq 3,5$)	juez consultivo	D11: la calidad también bloquea	reescrituras con presupuesto 2

Lo que todavía no está demostrado

Limitaciones

- **Evals «before»:** b2 y b3 quedan bloqueadas por brief_coverage; falta publicar la novela de 10 capítulos del ejemplo.
- **Lean:** aún sin un caso real cazado en pre_publish que no viera otro validador.
- **Reparto sin versionar:** solo el nombre se reconstruye por versión.
- **Verificación ligera (V3),** registrada como riesgo aceptado U.
- **policy_guard trabaja por patrones;** la defensa de fondo es BibleRepository.
- **Regla de fronteras heredada:** 21 accesos a ficheros fuera de los stores, pendientes de decisión humana.

Siguientes pasos

- **Tuning de cobertura:** cambio de prompt, ejecución «after» y comparación.
- **Novela de ejemplo** de 10 capítulos, publicada y exportada a PDF.
- **Revisión humana** de una novela completa y calibración del juez (compare.py).
- **Versionar el reparto** (descripción y rol por versión).
- **Subir un rol a Sonnet** solo si falla de forma repetida, con registro.
- **Vídeo de demo** del flujo completo.

LA DECISIÓN DE DISEÑO MÁS IMPORTANTE

Separar la verdad (story bible SQLite + validadores deterministas + Lean/TLA+) de la prosa generada, con un bucle acotado planner → writer → editor → juez que solo publica versiones que pasan todos los validadores.

ANEXOS

[anexo-arquitectura.pdf](#) · [anexo-tla-spec.pdf](#) · [anexo-lean-invariantes.pdf](#) · [anexo-evals-tabla.pdf](#) · [anexo-revision-humana.pdf](#)